Reviewer Pack — Minimal Order of Quasi-Platonic Solid Symmetry Groups and Fr…

Entry 730c65d82c60 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 03:55:14 UTC

Minimal Order of Quasi-Platonic Solid Symmetry Groups and Frege Proof Length Correlation

Entry ID: `730c65d82c60` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 03:55:14 UTC

1. Conjecture

Field A (mathematical branch): Quasi-Platonic Solid Theory **Field B** (complexity object): Frege Proof Complexity

Statement:

> The minimal order of the symmetry group of a quasi-platonic solid φ is linearly correlated with the Frege proof length of its associated CNF, such that the minimal order of the symmetry group (ord(φ)) = Θ(frege_proof_length(φ)), where frege_proof_length(φ) represents the minimum number of steps required to reduce φ to a contradiction using Frege's method.

Rationale (proposer's reasoning):

> Quasi-platonic solids exhibit highly symmetrical structures which could potentially encode computational complexity. If this symmetry translates into a correlation with Frege proof length, it may reveal new insights into the structure of proofs and the inherent difficulty of problems in PSPACE.

Taxonomy category: `QuasiPlatonicSolidSymmetryGroupOrder` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `cee4d43e19c4b3dd`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the 30 generated quasi-platonic solids have a correlation coefficient between their minimal order of symmetry group (ord(φ)) and Frege proof length (frege_proof_length(φ)) greater than or equal to 0.5, with an aggregate mean difference less than or equal to 3 steps.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

# Helper functions for linear algebra
def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for k in range(i+1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        factor = A[i][i]
        for j in range(n):
            A[i][j] /= factor

        for k in range(i+1, n):
            factor = A[k][i]
            for j in range(n):
                A[k][j] -= factor * A[i][j]

    # Back-substitute to get the solution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = A[i][-1]
        for j in range(i+1, n):
            x[i] -= A[i][j] * x[j]

    return x
```

```
def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = Fraction(0, 1)
    sign = Fraction(1, 1)
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += sign * A[0][j] * determinant(submatrix)
        sign *= -Fraction(1, 1)
    return det

def inverse(A):
    n = len(A)
    det_A = determinant(A)
    if det_A == Fraction(0, 1):
        raise ValueError("Singular matrix")

    adjoint = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[:i] + A[i+1:]]
            cofactor = determinant(submatrix)
            adjoint[j][i] = (-1) ** (i+j) * cofactor

    return matrix_multiply(adjoint, Fraction(1, det_A))

def identity_matrix(n):
    return [[Fraction(1 if i == j else 0, 1) for j in range(n)] for i in range(n)]

# Function to generate a random quasi-platonic solid
def generate_quasi_platonic_solid(seed):
    random.seed(seed)
    n = random.randint(5, 30)
    phi = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    return phi

# Function to construct the associated CNF
def construct_cnf(phi):
    n = len(phi)
    cnf = []
    for i in range(n):
        clause = [j+1 if phi[i][j] == 0 else -(j+1) for j in range(n)]
        cnf.append(clause)
```

```
    return cnf

# Function to compute the Frege proof length
def frege_proof_length(cnf):
    n = len(cnf)
    clauses = [set(clause) for clause in cnf]
    unit_clauses = {i+1: [] for i in range(n)}
    for i, clause in enumerate(cnf):
        if len(clause) == 1:
            unit_clauses[abs(clause[0])].append(i)

    stack = []
    while True:
        found_unit_clause = False
        for i in range(n):
            if unit_clauses[i+1]:
                j = unit_clauses[i+1].pop()
                if cnf[j][0] == i+1:
                    stack.append((j, 1))
                else:
                    stack.append((j, -1))
                found_unit_clause = True
                break

        if not found_unit_clause:
            return len(stack)

        while stack and stack[-1][1] == 1:
            j, sign = stack.pop()
            for k in range(n):
                if i+1 in clauses[k]:
                    clauses[k].remove(i+1)
                    if len(clauses[k]) == 1:
                        unit_clauses[abs(clauses[k][0])].append(j)

        if not any(len(clause) > 0 for clause in clauses):
            return len(stack)

# Function to compute the minimal order of the symmetry group
def symmetry_group_order(phi):
    n = len(phi)
    A = [[phi[i][j] ^ phi[i][k] ^ phi[j][k] for k in range(n)] for i in range(n) for j in range(i+
    A = [row[1:] for row in A]
    gaussian_elimination(A)
    rank = sum(1 for row in A if any(row))
    return 2 ** (n - rank)

# Function to run a single trial
def run_trial(seed: int) -> dict:
    phi = generate_quasi_platonic_solid(seed)
    cnf = construct_cnf(phi)
    ord_phi = symmetry_group_order(phi)
    frege_len = frege_proof_length(cnf)

    return {
        "metric_name": "symmetry_group_order_frege_length_correlation",
```

```
        "metric_value": abs(ord_phi - frege_len),
        "instances_tested": 1,
        "n_max": len(phi),
        "conjecture_holds": ord_phi == frege_len,
        "counterexample": ""
    }

# Main function to run trials
if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample=\"symmetry_group_order_frege_length_correlation\"
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79301027.py", line 172, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79301027.py", line 154, in run_trial
    ord_phi = symmetry_group_order(phi)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79301027.py", line 146, in symmetry_gro
    gaussian_elimination(A)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79301027.py", line 32, in gaussian_elim
    A[i][j] /= factor
    ~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means it did not complete its execution to provide a correlation coefficient and metric mean for the generated quasi-platonic solids. | next: Re-run the test ensuring that it completes without crashing and provides the required data to evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14266
2	propose	ollama_remote	glm4:latest	0	0	14956
3	preregistration	ollama_remote	glm4:latest	0	0	21351
4	novelty	ollama_remote	glm4:latest	0	0	18203
5	novelty	ollama_remote	glm4:latest	0	0	9511
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18516
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20501
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10110
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31450
10	judge	ollama_remote	glm4:latest	0	0	12504

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 171369 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/730c65d82c60.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/730c65d82c60.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/730c65d82c60.tar.gz` (if generated)